

Murilo Tomaz Gonzaga

Prof. Douglas Rodrigues

Métodos Numéricos Computacionais

02 de maio de 2025

Relatório do Trabalho prático 2 - Diferenças Finitas

Implementação:

Foi utilizada a linguagem de programação Python para implementar a ideia do programa. Entre suas inúmeras bibliotecas, utilizei **matplotlib**, **math**, **time** e **os**, que deram suporte a toda a construção do projeto, desde a implementação da interface gráfica até a facilitação na implementação das funções.

A principal ideia do projeto era implementar os métodos de diferenças finitas, com o objetivo de calcular a derivada de funções, aprendidos em aula. Ou seja, foram implementadas as diferenças finitas, regressivas, centrais e progressivas com o objetivo de encontrarmos a velocidade média no ponto específico com o tempo e a distância. Como solicitado na folha de requisição do projeto, todas essas implementações são gerenciadas por uma interface no CMD, com opções por meio de linhas de comando e escolha do usuário.

Imports e variáveis globais:

```
#variaveis globais => informações da tabela fornecida
times = [0, 3, 5, 8, 10, 13]
distances = [0, 225, 383, 623, 742, 993]
derivadas_regressão = [76.3073788546256, 76.1573788546256, 76.0573788546256, 75.9073788546256, 75.8073788546256, 75.6573788546256] #Derivadas obtidas usando o polinômio de regressão obtido pelo geogebra: f
(x) = -0.025x^2 + 76.3073788546256x - 0.1354625559661
```

Como dito anteriormente, *import* de bibliotecas python que foram extremamente úteis para a resolução do trabalho, foram utilizados dois vetores de inteiros para guardar as informações da tabela e que assim foi possível utilizar em cada método.

Utilizando a ferramenta da web chamada GeoGebra, foi possível plotar o gráfico e por meio de regressão de funções recebi a função derivada do problema que estamos trabalhando, com ela foi possível visualizar a derivada “exata” para cada ponto da nossa tabela.

Métodos Computacionais:

```
#methods
def diff_prog(i): #diferença finita progressiva
    if i < len(times) - 1:
        h = times[i + 1] - times[i]
        return (distances[i + 1] - distances[i]) / h
    return None

def diff_reg(i): #diferença finita regressiva
    if i > 0:
        h = times[i] - times[i - 1]
        return (distances[i] - distances[i - 1]) / h
    return None

def diff_cent(i): #diferença finita central
    if i > 0 and i < len(times) - 1:
        h = (times[i + 1] - times[i - 1]) / 2
        return (distances[i + 1] - distances[i - 1]) / (2 * h)
    return None
```

Este é o núcleo do projeto, pois implementa o algoritmo das diferenças finitas. A fórmula em si é relativamente simples e, devido à prática adquirida com os exercícios das listas, sua implementação não apresentou grandes dificuldades. Além disso, observa-se que as variações entre os métodos (progressivo, regressivo e central) são mínimas, uma vez que a única diferença reside nos pontos utilizados no cálculo — seja o ponto subsequente, o anterior ou ambos. Dessa forma, a diferença central pode ser interpretada como uma combinação das abordagens progressiva e regressiva.

Funções utilitárias:

```
#utils
def clear_terminal():
    os.system("cls" if os.name == "nt" else "clear") #limpa terminal

def escolha(): #menu interativo com o usuário
    print("Escolha qual método deseja aplicar na tabela disponível:")
    print("[1] - Métodos de Diferenças finitas")
    print("[0] - Sair")

    return int(input("Digite o numero correspondente: "))

def tabela(): #cópia da tabela do pdf, para melhor visualização do problema
    print("-" * 11, "TABELA", "-" * 11)
    print("-" * 30)
    print("| Tempo (s) | Distância (m) |")
    print("-" * 30)
    for t, d in zip(times, distances):
        print(f"| {t:<9} | {d:<14} |")
    print("-" * 30)
    print()
```

Nesta seção são implementadas funções auxiliares, ou seja, funções que serão usadas por outras funções ao longo do projeto.

Primeiramente temos a função responsável por limpar o terminal, que executa o comando adequado automaticamente, seja no terminal do Windows ou Unix.

Em seguida, são implementadas as duas funções principais que compõem minha interface de linha de comando. A função "escolha" atua como um seletor de ações, permitindo que o usuário escolha qual operação deseja realizar. Já a função "tabela" foi desenvolvida como parte da interface visual (UI), com o objetivo de apresentar de forma organizada as informações contidas no PDF disponibilizado pelo professor.

Funções ilustrativas:

```
def plot_grafico(data):
    #vetor de pontos para representar no grafico
    vp = [linha["vp"] for linha in data]
    vr = [linha["vr"] for linha in data]
    vc = [linha["vc"] for linha in data]

    plt.figure(figsize=(10, 5))
    plt.plot(times, vp, 'o-', label="Progressiva") #linha da progressiva, com bolinhas nos pontos
    plt.plot(times, vr, 'rs--', label="Regressiva") #linha da regressiva, com quadradinhos nos pontos
    plt.plot(times, vc, 'kx-.', label="Central") #linha da central, com X nos pontos

    plt.title("Velocidade Instantânea por Tempo", fontsize=14) #titulo
    plt.xlabel("Tempo (s)")
    plt.ylabel("Velocidade (m/s)")
    plt.grid(True)
    plt.legend() #atribuindo legenda para o grafico ficar mais informativo
    plt.tight_layout()
    plt.savefig("grafico_velocidade.png") #salvando imagem
    plt.close()
```

Utiliza-se a biblioteca Matplotlib para gerar um gráfico que representa a velocidade (em m/s) em função do tempo. Essa visualização facilita a identificação de variações e tendências nos valores obtidos pelos diferentes métodos de aproximação da derivada.

```
def relatorio(data):
    print("\nRELATORIO:\n")
    print("-" * 60)
    i = 0
    for linha in data:
        print(f"Tempo = {linha['tempo']} s")
        print(f"Regressiva: {linha['vr']:.2f} m/s || Erro : {abs(linha['vr'] - derivadas_regressao[i]):.4f}" if linha['vr'] else
              "Regressiva: N/A")
        print(f"Central: {linha['vc']:.2f} m/s || Erro : {abs(linha['vc'] - derivadas_regressao[i]):.4f}" if linha['vc'] else
              "Central: N/A")
        print(f"Progressiva: {linha['vp']:.2f} m/s || Erro : {abs(linha['vp'] - derivadas_regressao[i]):.4f}" if linha['vp'] else
              "Progressiva: N/A")
    print("-" * 60)
    i += 1
```

A segunda função imprime no terminal (cmd) um relatório textual com os dados obtidos, permitindo uma avaliação detalhada do desempenho dos métodos sem a necessidade de abrir os arquivos de imagem gerados pelo Matplotlib

```
def plot_tabelas(data):
    import matplotlib.gridspec as gridspec

    fig = plt.figure(figsize=(10, 6))
    spec = gridspec.GridSpec(nrows=2, ncols=1, height_ratios=[1, 1]) #usando o gridspec para definir o
    tamanho de colunas e linhas da nossa imagem

    #primeira tabela
    ax1 = fig.add_subplot(spec[0])
    ax1.axis('off')
    ax1.set_title("Velocidade Instantânea", fontsize=12, pad=5)

    col_labels = ["Tempo (s)", "Regressiva", "Central", "Progressiva"]
    table_data = [
        linha["tempo"],
        f"{linha['vr']:.2f}" if linha["vr"] is not None else "N/A",
        f"{linha['vc']:.2f}" if linha["vc"] is not None else "N/A",
        f"{linha['vp']:.2f}" if linha["vp"] is not None else "N/A",
    ]
    for linha in data:
        (variable) tabela1: Table
        tabela1 = ax1.table(cellText=table_data, colLabels=col_labels, loc='center', cellLoc='center')
        tabela1.auto_set_font_size(False)
        tabela1.set_fontsize(9)
        tabela1.scale(1, 1.2)
```

```
#segunda tabela
ax2 = fig.add_subplot(spec[1])
ax2.axis('off')
ax2.set_title("Erros Absolutos em Relação à Derivada Real", fontsize=12, pad=5)

col_labels2 = ["Tempo", "Erro R.", "Erro C.", "Erro P."]
table_data2 = [
    linha["tempo"],
    f"{abs(linha['vr'] - derivadas_regressão[i]):.4f}" if linha["vr"] is not None else "N/A",
    f"{abs(linha['vc'] - derivadas_regressão[i]):.4f}" if linha["vc"] is not None else "N/A",
    f"{abs(linha['vp'] - derivadas_regressão[i]):.4f}" if linha["vp"] is not None else "N/A",
]
for i, linha in enumerate(data):
    tabela2 = ax2.table(cellText=table_data2, colLabels=col_labels2, loc='center', cellLoc='center')
    tabela2.auto_set_font_size(False)
    tabela2.set_fontsize(9)
    tabela2.scale(1, 1.2)

plt.tight_layout()
plt.savefig("tabelas_velocidade_erro.png")
plt.close()
```

Além disso, na terceira função, são criadas duas tabelas: a segunda apresenta os erros absolutos associados a cada método de diferença finita (regressiva, central e

progressiva), enquanto a primeira organiza todos os valores calculados para uma análise completa do problema.

O erro é determinado pelo módulo da diferença entre o valor calculado por cada método e a derivada obtida por regressão, que é considerada uma referência e armazenada em uma variável global.

Main:

```
def main():
    e = 3
    tabela()
    e = escolha()
    while(e != 0 and e != 1):
        e = int(input("Digite um valor válido (1/0): "))
    if e == 1:
        clear_terminal()
        data = []
        for i in range (len(times)):
            line = {
                "tempo": times[i],
                "distancia": distances[i],
                "vp": diff_prog(i),
                "vr": diff_reg(i),
                "vc": diff_cent(i),
            }
            data.append(line)
        relatorio(data)
        plot_grafico(data)
        plot_tabelas(data)
    else:
        clear_terminal()
```

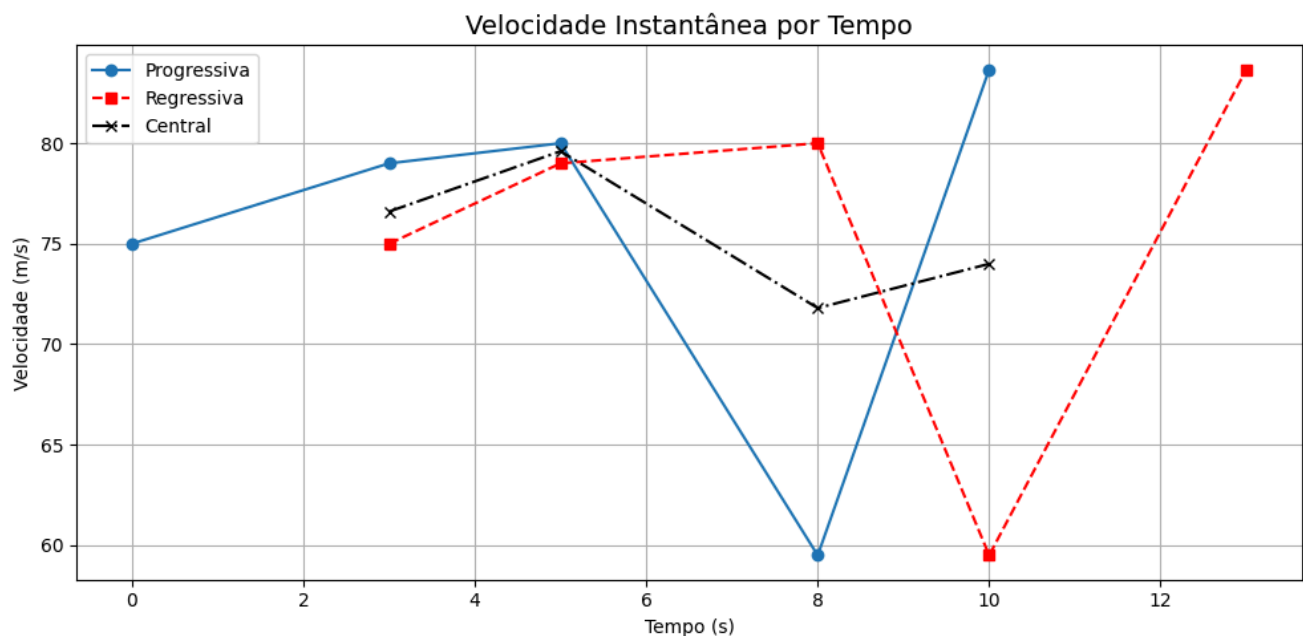
A função *main* é responsável por organizar a lógica geral do programa, integrando as demais funções e utilizando funções auxiliares previamente definidas. Nela, é realizado um loop para que a pessoa não digite um número diferente de 0 ou 1, que foram estabelecidos para sair ou iniciar o programa, evitando que o programa seja interrompido incorretamente. Portanto, o loop se repete até que o usuário digite um número válido. Logo depois, entramos na parte lógica da *main*, onde ela cria um vetor *data* para facilitar a passagem de dados para outras funções que necessitarão dos dados inteiros. Assim, ela *itera* no vetor de tempo para realizar os métodos. Em seguida, os métodos são executados, e os parâmetros e resultados são repassados para as funções ilustrativas, que se encarregam de gerar as representações gráficas.

Execução:

```
main()
```

Apenas realiza a inicialização da main.

Análises Gráficas:



Velocidade Instantânea

Tempo (s)	Regressiva	Central	Progressiva
0	N/A	N/A	75.00
3	75.00	76.60	79.00
5	79.00	79.60	80.00
8	80.00	71.80	59.50
10	59.50	74.00	83.67
13	83.67	N/A	N/A

Erros Absolutos em Relação à Derivada Real

Tempo	Erro R.	Erro C.	Erro P.
0	N/A	N/A	1.3074
3	1.1574	0.4426	2.8426
5	2.9426	3.5426	3.9426
8	4.0926	4.1074	16.4074
10	16.3074	1.8074	7.8593
13	8.0093	N/A	N/A

Nos gráficos plotados, fica nítido o motivo de não podermos utilizar as diferenças regressiva e central no primeiro valor da tabela: essas dependem de um valor anterior e, como se trata do valor inicial, não existe um valor anterior a ele. O mesmo ocorre no limite final, onde não é possível utilizar a diferença central e a progressiva, pois nessa iteração não há um valor posterior ao último, impossibilitando o uso desses métodos. Portanto, torna-se viável apenas o uso da diferença progressiva no início e da regressiva no final. É interessante perceber que a progressiva de uma célula será a regressiva da próxima célula, o que ocorre pela própria natureza das diferenças finitas.

Visualização do relatório:

```

RELATÓRIO:

-----
• Tempo = 0 s
  Regressiva: N/A
  Central: N/A
  Progressiva: 75.00 m/s || Erro : 1.3074)
-----
Tempo = 3 s
  Regressiva: 75.00 m/s || Erro : 1.1574)
  Central: 76.60 m/s || Erro : 0.4426)
  Progressiva: 79.00 m/s || Erro : 2.8426)
-----
Tempo = 5 s
  Regressiva: 79.00 m/s || Erro : 2.9426)
  Central: 79.60 m/s || Erro : 3.5426)
  Progressiva: 80.00 m/s || Erro : 3.9426)
-----
Tempo = 8 s
  Regressiva: 80.00 m/s || Erro : 4.0926)
  Central: 71.80 m/s || Erro : 4.1074)
  Progressiva: 59.50 m/s || Erro : 16.4074)
-----
Tempo = 10 s
  Regressiva: 59.50 m/s || Erro : 16.3074)
  Central: 74.00 m/s || Erro : 1.8074)
  Progressiva: 83.67 m/s || Erro : 7.8593)
-----
Tempo = 13 s
  Regressiva: 83.67 m/s || Erro : 8.0093)
  Central: N/A
  Progressiva: N/A
-----

```

CONCLUSÃO

Com a realização deste trabalho, ficou nítida a importância dos métodos computacionais aprendidos em sala de aula. A utilização e implementação desses métodos representam desafios, mas, com o algoritmo estruturado e a ideia bem compreendida, o processo se torna mais acessível. Além disso, a proposta de aplicar diversos algoritmos a diferentes funções e problemas torna a prática mais interessante

e contribui significativamente para o aprendizado, incentivando uma imersão profunda na linguagem Python.